

Comprehensive MyST Documnet

Mohammad Talebi-Kalaleh

Thursday 27th November, 2025

1 Getting Started

1.1 Setting Up Your Environment

First, create a project folder and set up a Python virtual environment. This keeps your project dependencies isolated and reproducible.

Bash/Shell

```
python -m venv .venv
```

Activate the environment:

- **Windows:** `.venv\Scripts\activate`
- **Mac/Linux:** `source .venv/bin/activate`

1.2 Installing Jupyter Book

Install the latest Jupyter Book 2.0 and the MyST extension for JupyterLab:

Bash/Shell

```
pip install "jupyter -book>=2.0.0a0"  
pip install jupyterlab -myst
```

1.3 Creating requirements.txt

For reproducibility, create a `requirements.txt` file with all your dependencies:

Bash/Shell

```
jupyter -book>=2.0.0a0  
jupyterlab -myst  
numpy  
matplotlib  
pandas  
scipy  
ipywidgets
```

Install all requirements:

Bash/Shell

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

2 Project Configuration

We'll organize content in a `content/` folder.

2.1 Understanding `myst.yml`

Creates a `myst.yml` configuration file. The `myst.yml` file controls your book's metadata and build settings. You can quickly generate a starter `myst.yml` configuration file by running the following command in your project folder:

Bash/Shell

```
myst init
myst init - -write -toc
```

This will interactively guide you through some basic questions, then create a `myst.yml` with sensible defaults tailored for your project. You can edit the generated file to further customize settings as shown below. Here's a minimal configuration for an academic paper:

YAML

```
## See docs at: https://mystmd.org/guide/frontmatter
version: 1
project:
  title: Your Paper Title
  description: A computational research paper
  keywords:
    - computational -mechanics
    - jupyter -book
  github: yourusername/your -repo
  # For a single paper, list your main file
  toc:
    - file: content/paper.md
site:
  template: article -theme # Use article -theme for papers
```

For a book with multiple chapters, adjust the `toc` section:

YAML

```
project:
  toc:
    - file: content/intro.md
    - file: content/chapter1.md
    - file: content/chapter2.md
site:
  template: book -theme # Use book -theme for books
```

2.2 Starting the Live Preview

Start the development server with automatic code execution:

Bash/Shell

```
.\.venv\Scripts\jupyter -book.exe start - --execute
```

The `--execute` flag is crucial because:

- It executes all code cells during the initial build
- It re-runs code cells automatically when you modify them
- Without it, notebooks with no outputs won't show any results

Your book will be available at <http://localhost:3000>. The page updates automatically as you edit files, and code cells re-execute on changes.

3 Writing Foundations & Visual Content

3.1 File Formats

Jupyter Book supports three main formats:

- **MyST Markdown** (`.md`) - Best for narrative content
- **Jupyter Notebooks** (`.ipynb`) - For code-heavy content
- **LaTeX** (`.tex`) - For complex mathematical documents

We'll focus on MyST Markdown as it's easiest to maintain and version control.

3.2 Default Frontmatter Configuration

Start each MyST markdown file with this frontmatter to enable proper execution and numbering:

YAML

```
- - -
jupytertext:
  text_representation:
    extension: .md
    format_name: myst
    format_version: 0.13
    jupytertext_version: 1.16.0
kernel_spec:
  name: python3
  display_name: Python 3 (ipykernel)
  language: python
numbering:
  headings: true
  code_cell: true
  figures: true
  tables: true
  equations: true
- - -
```

This configuration:

- **jupyter**: Ensures proper parsing of MyST markdown
- **kernelspec**: Specifies Python kernel for code execution
- **numbering**: Enables automatic numbering for all elements

You can toggle numbering for specific elements by setting them to **false**.

3.3 Document Structure

Create clear hierarchy with headings:

Markdown

```
# Chapter Title

## Main Section

### Subsection

#### Subsubsection

Regular paragraph text goes here.
```

3.4 Mathematics

Write inline math using single dollars: $E = mc^2$

For display equations with labels:

Markdown

```
$$
\frac{\partial u}{\partial t} + \nabla \cdot (u \otimes u) = -\nabla p + \nu \nabla^2 u \quad \leftarrow
\quad \leftarrow \nabla^2 u
$$ (eq:navier -stokes)
```

$$\frac{\partial u}{\partial t} + \nabla \cdot (u \otimes u) = -\nabla p + \nu \nabla^2 u \quad (1)$$

3.5 Figures

Add figures with captions and labels for cross-referencing:

Markdown

```
:::{figure} ../images/beam -deflection.svg
:label: fig:beam -fig
:align: center
:width: 60%

Deflection of a cantilever beam under point load
```

```
...
```

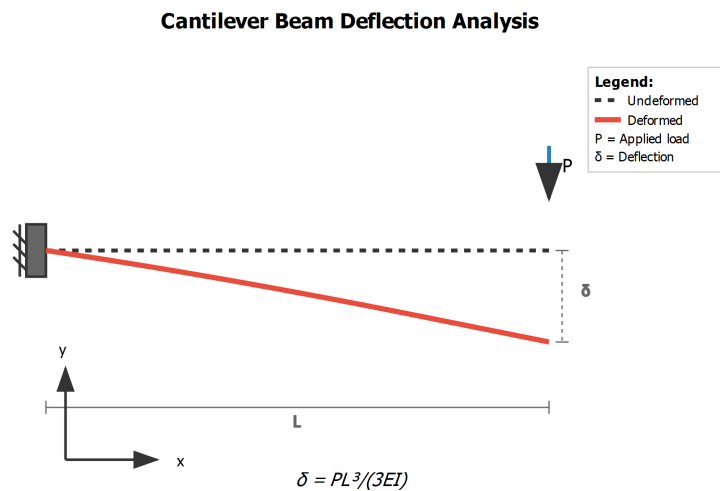


Figure 1: Deflection of a cantilever beam under point load

For subfigures, use a grid structure:

Markdown

```
::::{figure} ../images/comparison.svg
:label: fig:comparison -fig
:align: center

::::{image} ../images/experimental.svg
:::

::::{image} ../images/simulation.svg
:::

Experimental results
::::
```

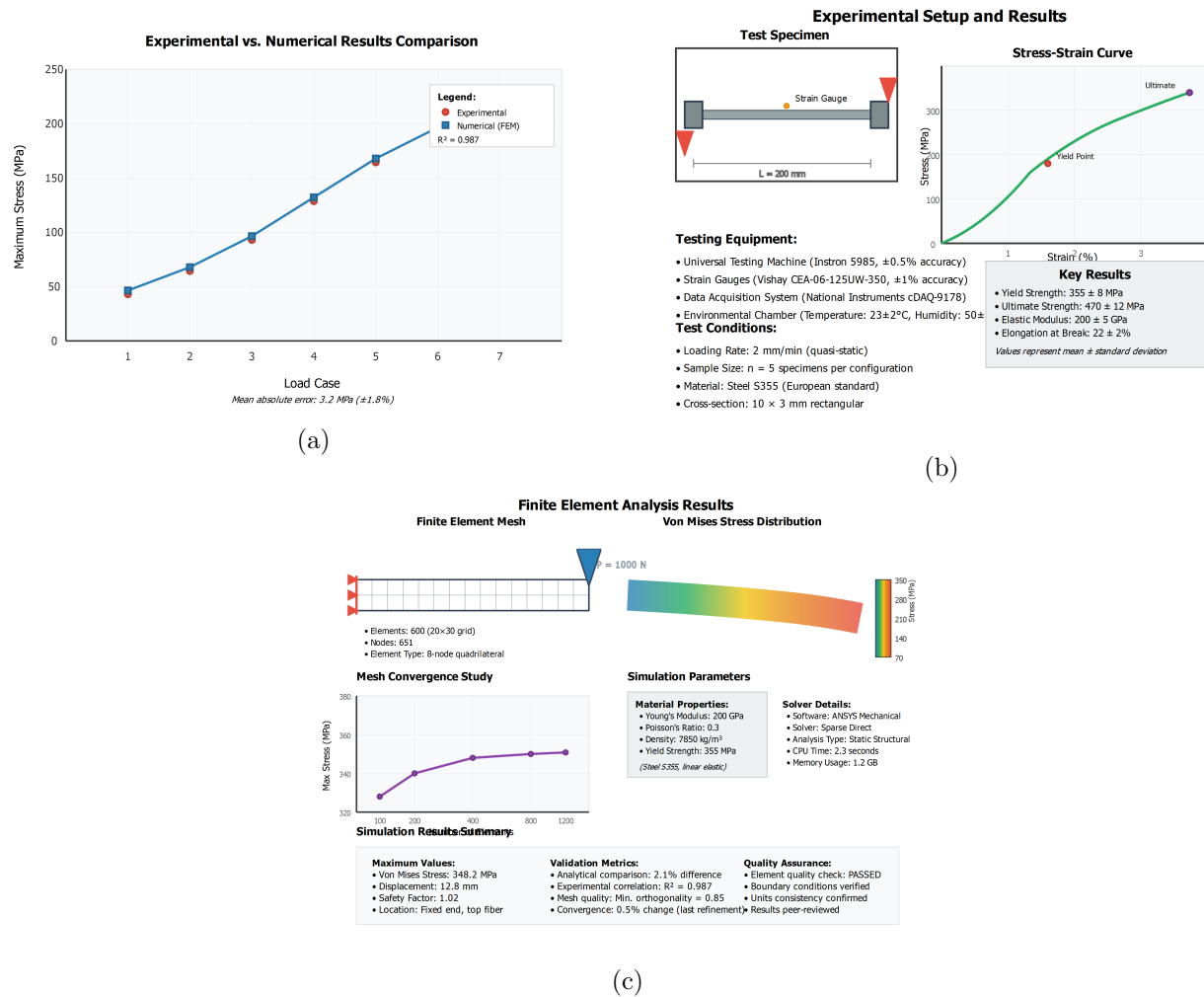


Figure 2: Experimental results

Tip

Save PowerPoint diagrams as SVG format for editability and scalability.

3.6 Tables

Simple markdown tables:

Markdown

```

:::{table} Material Properties
:label: tbl:material -table
:align: center
| Material | Young's Modulus (GPa) | Poisson's Ratio |
| --- | --- | --- |
| Steel | 200 | 0.30 |
| Aluminum | 69 | 0.33 |

```

```
| Concrete | 30 | 0.20 |
:::
```

Table 1: Material Properties

Material	Young's Modulus (GPa)	Poisson's Ratio
Steel	200	0.30
Aluminum	69	0.33
Concrete	30	0.20

For CSV data inline in your document:

Markdown

```
:::{csv -table} Material Properties
:label: tbl:material -table -csv
:align: center
:header: "Material", "Density (kg/m3)", "Yield Strength (MPa)"

"Steel", 7850, 250
"Aluminum", 2700, 95
"Titanium", 4500, 880
"Concrete", 2400, 3
:::
```

Table 2: Material Properties

Material	Density (kg/m ³)	Yield Strength (MPa)
Steel	7850	250
Aluminum	2700	95
Titanium	4500	880
Concrete	2400	3

For external table files, use the `{table}` directive with the `{include}` directive nested inside. The included file should contain HTML table markup:

Markdown

```
:::{table} Material Properties from CSV File
:label: tbl:material -table -file
:align: center

:::{include} ../data/materials.html
:::

::::
```

Table 3: Material Properties from CSV File

Material	Density (kg/m ³)	E (GPa)	Yield (MPa)	Ultimate (MPa)
Steel S355	7850	200	355	470
Aluminum 6061	2700	69	276	310
Concrete C30	2400	31	30	40
Titanium Grade 5	4430	114	880	950
Wood (Pine)	500	11	40	60

4 Scholarly Features

4.1 Cross-References

Reference any labeled element using its label:

As shown in Figure 1, the deflection increases linearly.

The governing Equation (1) describes fluid motion.

Material properties are listed in Table 1.

4.2 Numbered References

Use `{numref}` for numbered references with custom text:

Markdown

```
{numref}'fig:beam -fig'
{numref}'tbl:material -table'
{numref}'Equation %s <eq:navier -stokes>'
```

Figure 1 shows the beam deflection.

See Table 1 for material properties.

From (1), we can derive...

4.3 Citations

Method 1: BibTeX

Create a `references.bib` file:

Markdown

```
'''{code} bibtex
:filename: references.bib

@article{smith2023,
  title={Computational Mechanics of Structures},
  author={Smith, John and Doe, Jane},
  journal={Journal of Engineering},
  year={2023}
}
'''
```

BibTeX

```
@article{smith2023,
  title={Computational Mechanics of Structures},
  author={Smith, John and Doe, Jane},
  journal={Journal of Engineering},
  year={2023}
}
```

Add to frontmatter and cite:

YAML

```
- - -
bibliography: references.bib
- - -
```

Markdown

```
[@smith2023]
[@smith2023; @zhang2023]
```

Recent studies [[Smith and Doe, 2023](#)] show that...

Multiple citations [[Smith and Doe, 2023](#), [Zhang et al., 2023](#)] indicate...

Method 2: DOI

Cite directly using DOI:

Markdown

```
@10.1016/j.jsv.2025.119289
```

The foundational work [Talebi-Kalaleh et al. \[2025\]](#) established...

4.4 Abbreviations

Define abbreviations project-wide in `myst.yml` (recommended) or in individual files:

YAML

```
## In myst.yml - applies to all pages
project:
  title: Structural Analysis Guide
  authors:
    - name: Mohammad Talebi -Kalaleh
      affiliations:
        - University of Alberta
  abbreviations:
    SHM: Structural Health Monitoring
    FEM: Finite Element Method
```

```
DOF: Degrees of Freedom
FFT: Fast Fourier Transform
PCA: Principal Component Analysis
```

Then use abbreviations anywhere in your content:

The [FEM](#) uses [DOF](#) to analyze structures. [SHM](#) systems often employ [FFT](#) for signal processing.

4.5 Frontmatter Metadata

You can set metadata in two places:

Option 1: Project-wide in `myst.yml` (recommended for common metadata):

YAML

```
## In myst.yml
project:
  title: Computational Mechanics Book
  description: Advanced structural analysis techniques
  keywords: [structural analysis, computational mechanics, finite elements]
  authors:
    - name: Mohammad Talebi -Kalaleh
      affiliations:
        - University of Alberta
      orcid: 0009 -0004 -3754 -5992
```

Option 2: Page-specific in `.md` files (for unique page metadata):

YAML

```
- - -
title: Chapter 3: Nonlinear Analysis
date: 2024 -11 -19
doi: 10.1000/chapter -doi
# Page -specific metadata overrides project defaults
- - -
```

Best practice: Define common metadata (authors, keywords, abbreviations) in `myst.yml`, then add page-specific details (title, date, DOI) in individual files.

5 Code Integration

5.1 Global Configuration

Create a hidden setup cell at the document start:

Markdown

```
'''{code -cell} python
:tags: [remove -cell]
'''
```

5.2 Static Code Blocks

Display code without execution:

Markdown

```
'''python
'''
```

Python

```
def calculate_stress(force, area):
    """Calculate engineering stress"""
    return force / area
```

With line numbers and highlighting:

Markdown

```
'''{code} python
:linenos:
:emphasize -lines: 2
'''
```

Python

```
def beam_deflection(load, length, E, I):
    # Maximum deflection for cantilever beam
    delta_max = (load * length**3) / (3 * E * I)
    return delta_max
```

5.3 Executable Code Cells

Create executable cells that run when building:

Markdown

```
'''{code -cell} python
:caption: Figure Generated by Python Code Cell with Caption.
:label: fig:python -plot
'''
```

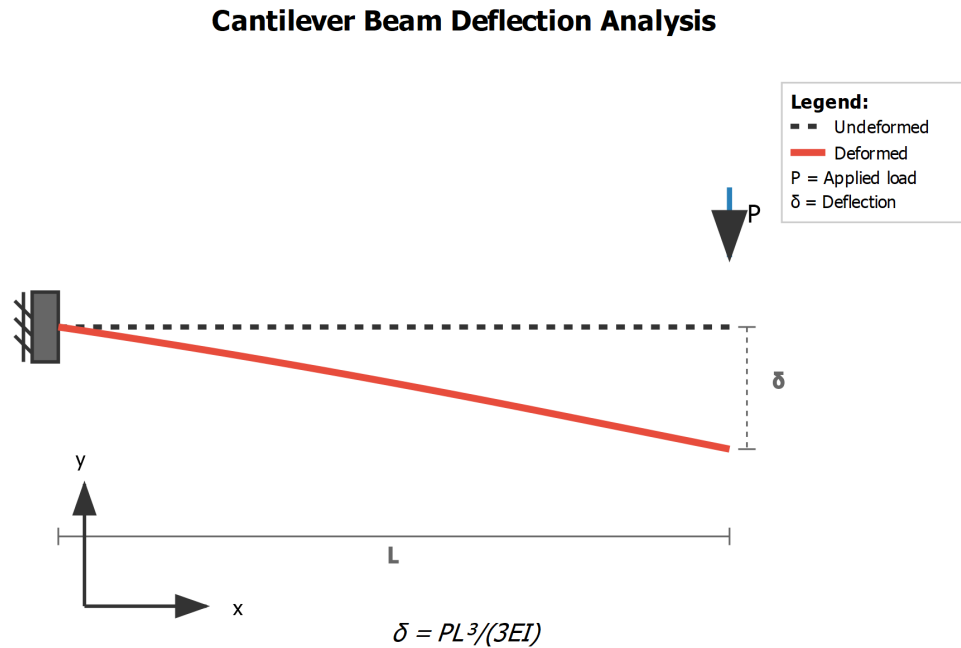


Figure 3: Figure Generated by Python Code Cell with Caption.

Create and display tables using pandas:

Python

```
import pandas as pd
from IPython.display import display

# Define material properties
data = {
    "Material": ["Steel", "Aluminum", "Concrete"],
    "Density (kg/m3)": [7850, 2700, 2400],
    "Yield Strength (MPa)": [250, 95, 3]
}

# Create a DataFrame
material_df = pd.DataFrame(data)

display(material_df.style.hide(axis='index').set_properties(**{'text-align':
    'left'}))
```

5.4 Including External Code

Include code from separate files. This directive is helpful for showing code snippets without duplicating your content:

Markdown

```
‘‘{literalinclude} ../scripts/analysis.py
:lineno -match:
```

```
'''
```

Python

```
'''
Structural Analysis Module
Example code for Jupyter Book Workshop
'''

import numpy as np

def calculate_beam_deflection(load, length, E, I):
    """Calculate maximum deflection of a simply supported beam"""
    return (5 * load * length**4) / (384 * E * I)

def natural_frequency(mass, stiffness):
    """Calculate natural frequency of SDOF system"""
    return np.sqrt(stiffness / mass) / (2 * np.pi)

def stress_analysis(force, area):
    """Calculate normal stress"""
    return force / area

if __name__ == "__main__":
    # Example usage
    deflection = calculate_beam_deflection(10, 5, 200e9, 1e-4)
    print(f"Maximum deflection: {deflection*1000:.2f} mm")
```

Markdown

```
'''{literalinclude} ../scripts/analysis.py
:start -at: def calculate_beam_deflection
:end -before: def natural_frequency
:lineno -match:

'''
```

Python

```
def calculate_beam_deflection(load, length, E, I):
    """Calculate maximum deflection of a simply supported beam"""
    return (5 * load * length**4) / (384 * E * I)
```

5.5 Controlling Cell Visibility

Use tags to control what readers see:

Available tags:

- `hide-input`: Collapse code (can be expanded)
- `hide-output`: Collapse output

- `remove-input`: Completely remove code
- `remove-output`: Completely remove output
- `remove-cell`: Remove entire cell

First, let's define the required functions, then let's use different cell tags:

Markdown

```
'''{code -cell} python
:tags: [remove -cell]

'''
```

Markdown

```
'''{code -cell} python
:tags: [hide -input]

'''
```

Markdown

```
'''{code -cell} python
:tags: [hide -output]

'''
```

Python

```
## Complex calculation - output hidden
calculation_results = []
for i in range(1000):
    result = intermediate_calc(i)
    calculation_results.append(result)
print(f"Completed {len(calculation_results)} calculations")
```

6 Enhanced Content

6.1 Inline Code Results with `{eval}`

Display variables from code cells inline:

Markdown

```
{eval}'max_stress'
{eval}'safety_factor'
```

The maximum stress is `max_stress` MPa, with a safety factor of `safety_factor`.

6.2 Interactive Widgets

Create interactive visualizations and controls using ipywidgets:

Python

```
import ipywidgets as widgets
from IPython.display import display

@widgets.interact(frequency=(1, 10, 0.5))
def plot_wave(frequency=2):
    x = np.linspace(0, 2*np.pi, 100)
    y = np.sin(frequency * x)
    plt.plot(x, y)
    plt.ylim(-1.5, 1.5)
    plt.xlabel('x')
    plt.ylabel('sin(fx)')
    plt.title(f'Sine Wave: f = {frequency}')
    plt.show()
```

Advanced Widget Dashboard:

Python

```
import ipywidgets as widgets
from IPython.display import display

# Text widgets
text_input = widgets.Text(
    value='Hello World',
    placeholder='Type something',
    description='Text Input:',
    disabled=False
)

textarea = widgets.Textarea(
    value='Multi -line\ntext input',
    placeholder='Type something',
    description='Text Area:',
    disabled=False
)

# Numeric widgets
int_slider = widgets.IntSlider(
    value=7,
    min=0,
    max=10,
    step=1,
    description='Int Slider:'
)

float_slider = widgets.FloatSlider(
    value=7.5,
    min=0,
    max=10.0,
```

```
        step=0.1,
        description='Float Slider:'
    )

    int_range_slider = widgets.IntRangeSlider(
        value=[5, 7],
        min=0,
        max=10,
        step=1,
        description='Range:'
    )

    # Selection widgets
    dropdown = widgets.Dropdown(
        options=['Option 1', 'Option 2', 'Option 3'],
        value='Option 1',
        description='Dropdown:'
    )

    radio_buttons = widgets.RadioButton(
        options=['Option A', 'Option B', 'Option C'],
        description='Radio:'
    )

    select_multiple = widgets.SelectMultiple(
        options=['Apple', 'Banana', 'Cherry', 'Date'],
        value=['Apple'],
        description='Multi -Select:'
    )

    # Boolean widgets
    checkbox = widgets.Checkbox(
        value=False,
        description='Checkbox',
        disabled=False
    )

    toggle_button = widgets.ToggleButton(
        value=False,
        description='Toggle Me',
        disabled=False,
        button_style='info',
        tooltip='Click me'
    )

    # Button widgets
    button = widgets.Button(
        description='Click Me!',
        disabled=False,
        button_style='success',
        tooltip='Click me',
        icon='check'
    )

    # Date and time widgets
    date_picker = widgets.DatePicker(
        description='Pick a Date',
        disabled=False
```

```

)

# Color picker
color_picker = widgets.ColorPicker(
    concise=False,
    description='Pick a color',
    value='blue',
    disabled=False
)

# File upload
file_upload = widgets.FileUpload(
    accept='', # Accepted file extension
    multiple=False # True to accept multiple files
)

# Progress bar
progress = widgets.IntProgress(
    value=7,
    min=0,
    max=10,
    description='Loading:',
    bar_style='info',
    style={'bar_color': 'maroon'},
    orientation='horizontal'
)

# Display all widgets in organized layout
display(widgets.HTML('<h3>Widget Types Demonstration</h3>'))

display(widgets.HTML('<h4>Text Input Widgets</h4>'))
display(widgets.HBox([text_input, textarea]))

display(widgets.HTML('<h4>Numeric Widgets</h4>'))
display(widgets.VBox([int_slider, float_slider, int_range_slider]))

display(widgets.HTML('<h4>Selection Widgets</h4>'))
display(widgets.HBox([
    widgets.VBox([dropdown, radio_buttons]),
    select_multiple
]))

display(widgets.HTML('<h4>Boolean and Button Widgets</h4>'))
display(widgets.HBox([checkbox, toggle_button, button]))

display(widgets.HTML('<h4>Specialized Widgets</h4>'))
display(widgets.HBox([date_picker, color_picker]))
display(widgets.VBox([file_upload, progress]))

# Add event handling example
def on_button_clicked(b):
    with widgets.Output():
        print("Button clicked!")

button.on_click(on_button_clicked)

```

6.3 Admonitions

Highlight important information:

Markdown

```
:::{note}  
This method assumes linear elastic behavior.  
:::
```

Note

This method assumes linear elastic behavior.

Markdown

```
:::{attention}  
Ensure all units are consistent before performing calculations.  
:::
```

Attention

Ensure all units are consistent before performing calculations.

Markdown

```
:::{important}  
Safety factors must comply with local building codes.  
:::
```

Important

Safety factors must comply with local building codes.

Markdown

```
:::{caution}  
Avoid using this formula for non-linear materials.  
:::
```

Caution

Avoid using this formula for non-linear materials.

Markdown

```
 :::{hint}  
 Consider using a numerical solver for complex systems.  
 :::
```

Hint

Consider using a numerical solver for complex systems.

Markdown

```
 :::{warning}  
 Check boundary conditions before applying this formula.  
 :::
```

Warning

Check boundary conditions before applying this formula.

Markdown

```
 :::{seealso}  
 Refer to the appendix for detailed derivations.  
 :::
```

See Also

Refer to the appendix for detailed derivations.

Markdown

```
 :::{danger}  
 Incorrect application of this method can lead to structural failure.  
 :::
```

Danger

Incorrect application of this method can lead to structural failure.

Markdown

```
 :::{tip}  
 :class: dropdown
```

```
Use dimensionless parameters to generalize your results.
:::
```

Tip

Use dimensionless parameters to generalize your results.

Markdown

```
:::{error}
Calculation error detected: please review input values.
:::
```

Error

Calculation error detected: please review input values.

6.4 Exercise and Solution Blocks

Create teaching materials:

Markdown

```
:::{exercise}
:label: ex1
Calculate the natural frequency of a cantilever beam with:

- Length L = 2 m
- Mass m = 10 kg at the tip
- Flexural rigidity EI = 1000 N m2
:::
```

Markdown

```
:::{solution} ex1
:class: dropdown

Using the formula for a cantilever with tip mass:

$$\omega_n = \sqrt{\frac{3EI}{mL^3}}$$


Substituting values:

$$\omega_n = \sqrt{\frac{3 \times 1000}{10 \times 2^3}} = 6.12 \text{ rad/s}$$


:::
```

Markdown

```

:::{exercise -start}
:label: ex2
:::

‘‘{code -cell} python
:tags: [remove -cell]

‘‘‘

:::{exercise -end}
:::

```

Markdown

```

:::{solution -start} ex2
:class: dropdown
:::

‘‘{code -cell} python
:tags: [remove -cell]

‘‘‘

:::{solution -end}
:::

```

6.5 Mermaid Diagrams

Create flowcharts and diagrams:

Markdown

```

‘‘{mermaid}
flowchart TD
    A[Load Applied] --> B[Linear Analysis]
    B -->|Small Deformation| C[Linear Solution]
    B -->|Large Deformation| D[Nonlinear Analysis]
    D --> E[Iterative Solution]
    E --> F[Converged?]
    F -->|No| E
    F -->|Yes| G[Final Results]
‘‘‘

```

6.6 Videos and iFrames

Embed multimedia content:

Markdown

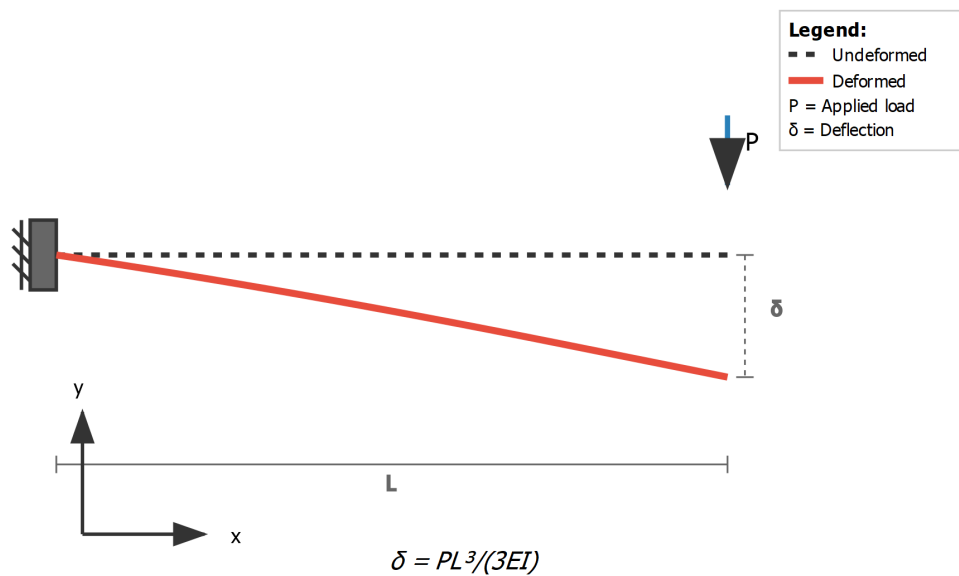
```

:::{iframe} ↩
    ↩ https://www.youtube.com/embed/CRa9djnZRu0?start=511&end=655&version=3&rel=0&autoplay=0
:width: 100%
Sample Youtube Video

:::

```

Cantilever Beam Deflection Analysis



Sample Youtube Video

Markdown

```

:::{figure} ../videos/experiment.mp4
:label: fig:video -local
:width: 80%
Sample Local Video: Experimental setup and testing procedure
:::

```

Figure 4: Sample Local Video: Experimental setup and testing procedure

7 Exporting to LaTeX and PDF

7.1 List Available Templates

View journal templates:

Code

```
jupyter -book templates list - -pdf
```

Common Academic Templates:**1. Plain LaTeX Book (plain_latex_book)**

- Description: A plain latex book theme
- Tags: book
- Best for: Traditional academic books and comprehensive documentation

2. LaPreprint Typst Template (lapreprint-typst)

- Description: Easily create beautiful preprints in Typst
- Tags: preprint, article, paper
- Best for: Academic preprints and research papers

3. arXiv (NIPS Style) (arxiv_nips)

- Description: An arXiv compatible template based on the NIPS 2018 Style
- Tags: paper, preprint, arxiv, bioarxiv, eartharxiv
- Best for: Conference papers and arXiv submissions

7.2 Configure the Exports

To define exports and templates in your `myst.yml` file, you can specify the desired output format and template under the `exports` section. This allows you to customize the export settings for your Jupyter Book. Here is an example configuration:

YAML

```
project:
  exports:
    - format: pdf
      template: lapreprint -typst
      articles: [content/intro.md, content/chapter1.md]
      line_numbers: true
```

This configuration will:

- Export to PDF format using the `lapreprint-typst` template
- Include only the `intro.md` file
- Save outputs to the default `_build` directory structure

7.3 Build PDF

Generate PDF using default template:

Code

```
jupyter -book build - -pdf
```

Warning

The default PDF renderer uses $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ to create PDFs, which means that to work locally you will need to [Install LaTeX](#). A warning will occur if MyST cannot find a $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ environment, as well as forcing the build process and reporting any errors.

As an alternative, for faster PDF builds, you may use [Install Typst](#) instead.

textbfFor SVG graphics processing, install Inkscape:

beginitemize item **Windows:** `winget install Inkscape.Inkscape` or download from inkscape.org item **macOS:** `brew install inkscape` or download from inkscape.org enditemize

textbfHandling Animated Images Animated images, such as GIFs, are not well supported by the PDF format. When exporting to PDF, MyST will convert the first frame of an animated image to a static image. To perform this conversion, you need to have [ImageMagick](#) installed on your system. Here are the installation instructions for different operating systems:

beginitemize item **Windows:** `winget install ImageMagick.ImageMagick` item **macOS:** `brew install imagemagick` item **Linux:** Use your package manager, e.g., `sudo apt-get install imagemagick` enditemize

7.4 Export to LaTeX

For journal submission, export LaTeX source:

Code

```
jupyter -book build - -tex
```

The LaTeX files will be in `_build/tex/`.

8 Authoring with JupyterLab

8.1 Accessing the Jupyter Server

When you run `jupyter book start --execute`, it automatically starts a Jupyter server. You can access JupyterLab for a better authoring experience:

1. Look for the server URL in your terminal (typically `http://localhost:8888/lab`)
2. Open this URL in your browser to access JupyterLab

8.2 Live MyST Markdown Rendering

JupyterLab with the MyST extension provides real-time rendering:

Bash/Shell

```
## The MyST extension was installed earlier with:
pip install jupyterlab -myst
```

Benefits of authoring in JupyterLab:

- **Live preview:** Changes render immediately as you type
- **Execute cells:** Run code blocks directly in the editor
- **Better for teaching:** Show both source and output simultaneously

8.3 MyST vs Jupyter Notebooks

Why use MyST Markdown (.md) instead of Jupyter Notebooks (.ipynb)?

Aspect	MyST Markdown	Jupyter Notebook
Version Control	Clean diffs, easy merge	Binary format, merge conflicts
Maintenance	Plain text, find/replace works	JSON structure, harder to edit
Collaboration	Any text editor works	Requires Jupyter
File Size	Compact	Large with outputs
Reproducibility	Outputs generated on build	Outputs stored in file

8.4 Workflow Tips

1. **Author content:** Write in JupyterLab with live preview
2. **Test execution:** Code cells run as you write
3. **Version control:** Commit clean .md files without outputs
4. **Build outputs:** Generate fresh outputs with `--execute`

This approach separates content from presentation, making your book maintainable long-term.

9 Web Deployment with GitHub Pages

9.1 Create .gitignore

Code

```
:caption: .gitignore
.venv/
_build/
.ipynb_checkpoints/
__pycache__/
*.pyc
.DS_Store
```

9.2 GitHub Actions Workflow

Create `.github/workflows/deploy.yml`:

Markdown

```
on:
  push:
    branches: [main]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }

    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Build book
        run: |
          jupyter -book build - -html

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: '_build/html'

      - name: Deploy to GitHub Pages
        uses: actions/deploy-pages@v4
```

9.3 Enable GitHub Pages

1. Push your code to GitHub
2. Go to Settings → *Pages* Select “*GitHub Actions*” as source
3. Your book deploys automatically on each push

10 Maintenance & Collaboration

10.1 Version Control Workflow

Bash/Shell

```
## Clone repository
git clone https://github.com/yourusername/your -book.git
cd your -book

# Create feature branch
git checkout -b add -chapter -3

# Make changes and commit
git add .
git commit -m "Add analysis chapter"

# Push and create pull request
git push origin add -chapter -3
```

10.2 Collaborative Authoring

1. Each author works on separate branches
2. Use pull requests for review
3. Maintain consistent style with shared configuration

10.3 AI-Assisted Writing

Use VS Code with GitHub Copilot for:

- Auto-completing markdown text
- Generating boilerplate code cells
- Creating consistent documentation

10.4 Best Practices

1. **Keep content modular:** One chapter per file
 2. **Version control data:** Store small datasets in `data/`
 3. **Document dependencies:** Always update `requirements.txt`
 4. **Test builds locally:** Run `jupyter-book build` before pushing
 5. **Use semantic commits:** Clear commit messages for collaboration
-

11 Quick Reference

11.1 Essential Commands

Bash/Shell

```
jupyter book init           # Initialize project
jupyter book start - --execute # Start dev server with code execution
jupyter book build - -pdf    # Build PDF
jupyter book clean          # Clean build files
```

11.2 File Structure

Code

```
my -book/
+---- myst.yml           # Configuration
+---- requirements.txt    # Dependencies
+---- content/           # All content files
|   +---- intro.md        # First page
|   +---- chapter1.md     # Content chapters
|   +---- chapter2.md
|   +---- references.bib  # Bibliography
+---- data/              # Data files
+---- images/            # Figures
+---- scripts/           # External code
+---- _build/            # Build output (ignored)
+---- .github/
|   +---- workflows/
|       +---- deploy.yml  # GitHub Actions
```

11.3 Resources

- Documentation: <https://next.jupyterbook.org/tutorial>
- MyST Syntax: <https://mystmd.org/guide>
- Templates: <https://github.com/jupyter-book/cookiecutter-jupyter-book>

Bibliography

- J. A. Smith and J. B. Doe. Advanced Finite Element Methods in Structural Analysis. *Journal of Computational Mechanics*, 45(3):234–256, 2023.
- M. Talebi-Kalaleh, M. Gül, and Q. Mei. Crowdsensing-based bridge vibration monitoring using a sparse network of random mobile sensors: Theory and numerical verifications. *Journal of Sound and Vibration*, 618:119289, 12 2025. ISSN 0022-460X. doi: 10.1016/j.jsv.2025.119289. URL <http://dx.doi.org/10.1016/j.jsv.2025.119289>.
- W. Zhang, M. Liu, and L. Chen. Recent Advances in Structural Health Monitoring. *Engineering Structures*, 285:115234, 2023. doi: 10.1016/j.engstruct.2023.115234.
- main